INSTITUTO NEBRIJA | Formación Profesional

PORTADA

Actividad Individual 1

Desarrollo de Interfaces — DAM

ABSTRACT

En esta memoria documento DAM Gestión, una aplicación Ionic y Angular conectada a Supabase. He implementado acceso por roles, catálogo, formulario con validación asíncrona, componentes reutilizables, dashboard e informes PDF y CSV. Recojo las pruebas, las evidencias de la interfaz y la preparación de la entrega.

Nombre del alumno: Lucas Aja

ÍNDICE PAGINADO

Justificación del proyecto 3

Introducción 4

Objetivos y desarrollo de la aplicación 5

Componentes reutilizables 15

Catálogo, informes y diseño adaptable 17

Verificación de acceso y calidad 19

Edición de productos 22

Publicación y enlaces de entrega 23

Conclusiones 24

Bibliografía 25

Anexos 26

Otros puntos 27

1. JUSTIFICACIÓN DEL PROYECTO

He elegido una aplicación de inventario porque permite resolver una necesidad habitual de una empresa: conocer qué productos tiene, cuántas unidades quedan y cuál es su valor. Con este caso puedo trabajar formularios, componentes reutilizables, permisos y generación de informes en una misma solución.

He utilizado datos sintéticos de material informático. La información de negocio se almacena en Supabase y las credenciales de evaluación se entregarán al docente de forma privada.

2.

3. INTRODUCCIÓN

DAM Gestión es una aplicación web híbrida desarrollada con Ionic y Angular. Organizo el código en páginas, componentes y servicios: las plantillas presentan los datos, las páginas gestionan el estado y los servicios encapsulan el acceso a Supabase y la generación de informes.

La cuenta administradora registra productos y accede al dashboard; la cuenta estándar consulta el catálogo. La API REST de Supabase proporciona los datos relacionales y una fuente JSON simula documentos actualizados periódicamente. El objetivo es documentar una aplicación funcional y verificable, desde su ejecución local hasta su publicación.

4.

5. OBJETIVOS

A. OBJETIVO GENERAL

Mi objetivo es desarrollar y comprobar una aplicación de inventario con interfaz adaptable, acceso por roles, consulta de datos e informes, documentando su evolución y sus evidencias.

B. OBJETIVOS ESPECÍFICOS

Me propongo separar páginas, componentes y servicios; conectar el catálogo con Supabase; validar formularios; representar métricas; generar PDF y CSV; comprobar permisos y accesibilidad; y preparar la documentación, el repositorio y la publicación.

4. DESARROLLO

1. FUNDAMENTACIÓN TEÓRICA

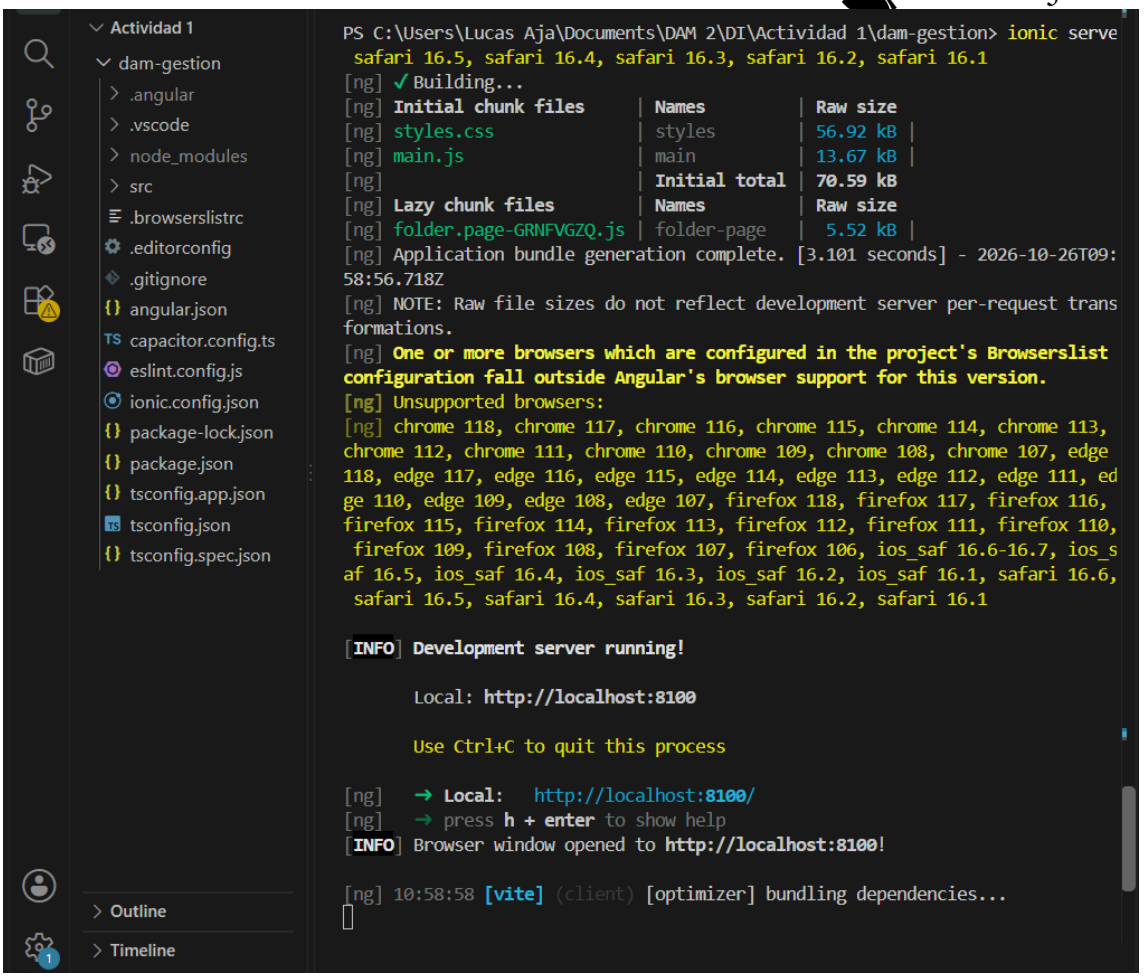
Utilizo Ionic para trabajar en la interfaz. Con ionic serve inicio el servidor de desarrollo para acceder al proyecto desde el navegador.

2. Materiales y métodos

Trabajo en Visual Studio Code y utilizo la terminal para ejecutar los comandos del proyecto. Registro cada paso con una captura y una explicación breve en primera persona.

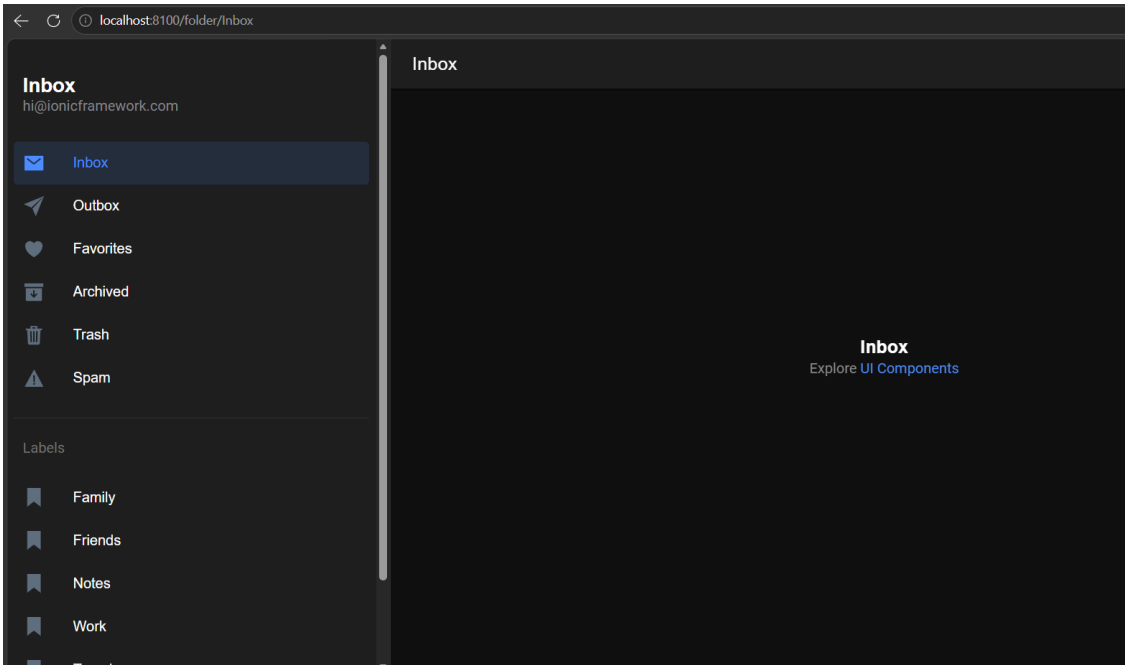
3. Resultados y análisis

Primera ejecución de Ionic



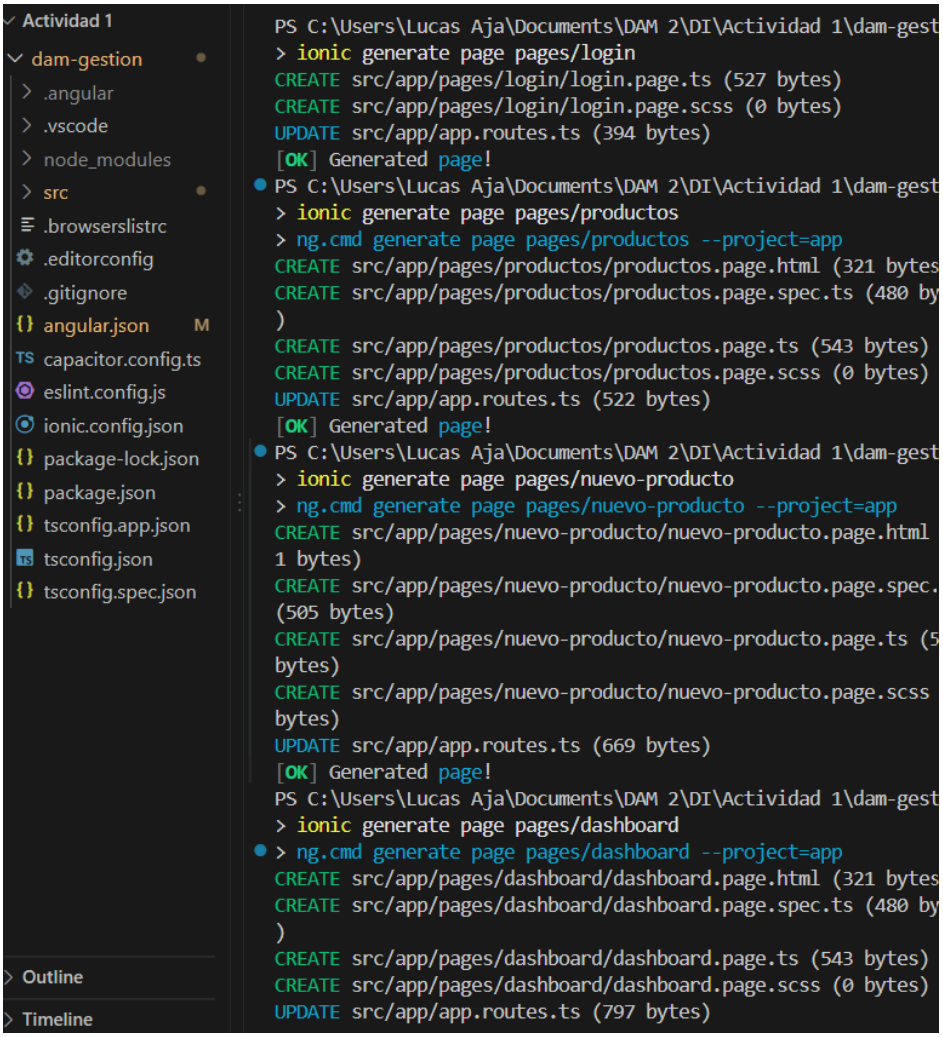
He ejecutado ionic serve desde la carpeta dam-gestion. La compilación ha finalizado y el servidor se ha iniciado en http://localhost:8100. Han aparecido avisos de compatibilidad con algunos navegadores, pero el servidor está funcionando.

Comprobación en localhost

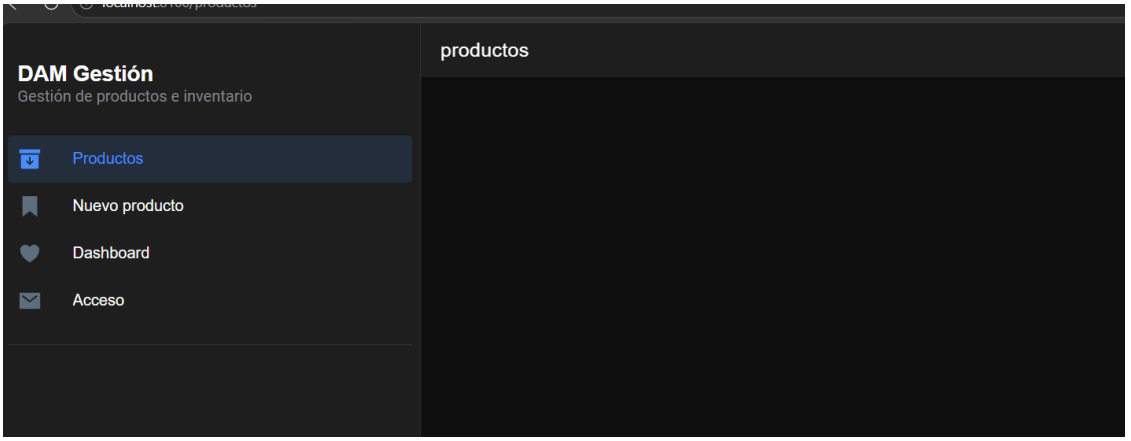


He accedido a http://localhost:8100/folder/Inbox y he comprobado que se muestra la pantalla inicial de Ionic, con el menú lateral y la sección Inbox.

Creación de las páginas principales



He generado las páginas login, productos, nuevo-producto y dashboard con Ionic CLI. Los comandos han creado los archivos de cada página y han actualizado app.routes.ts.



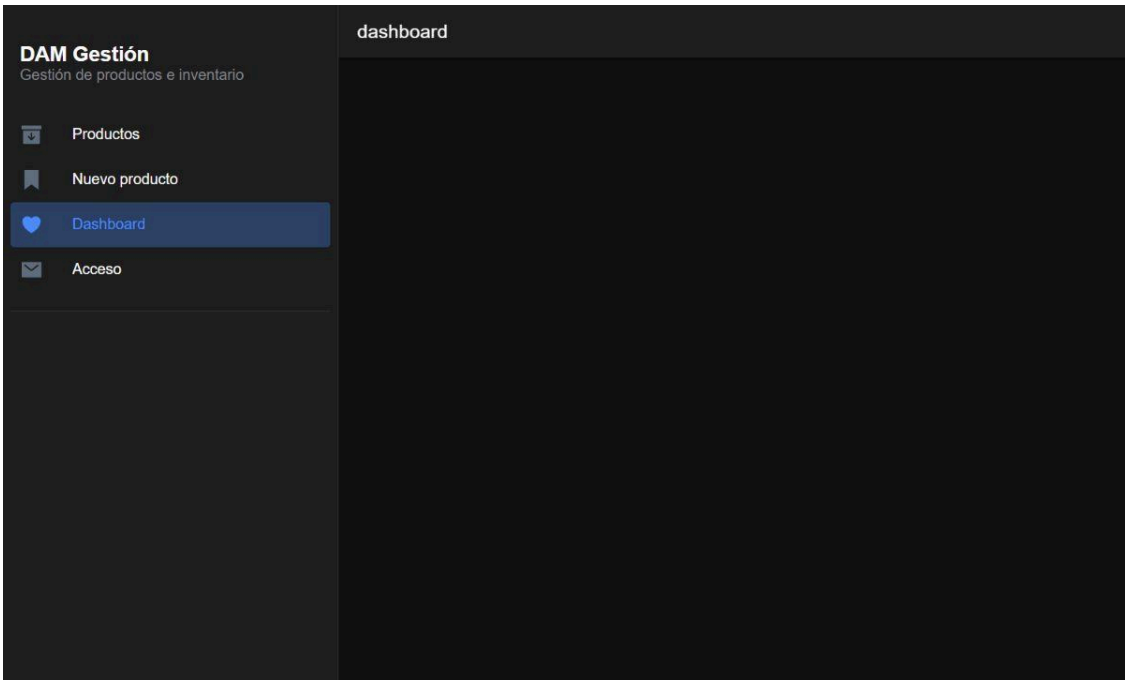
He sustituido el menú de ejemplo por las opciones Productos, Nuevo producto, Dashboard y Acceso. He comprobado que la página Productos se muestra en la ruta /product**os.**

Pantalla de inicio de sesión

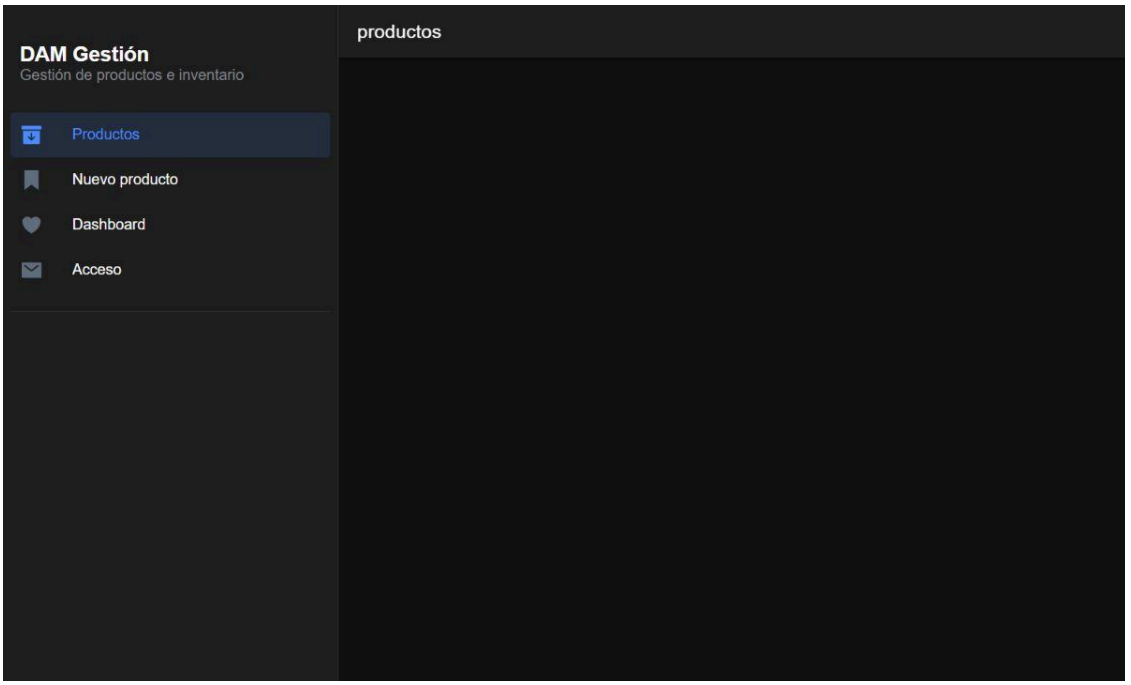


He creado el formulario de acceso con los campos de correo electrónico y contraseña.

Prueba de acceso como administrador



He iniciado sesión con la cuenta de prueba de administrador y he comprobado que puedo abrir la página Dashboard. Su contenido está pendiente de desarrollar.



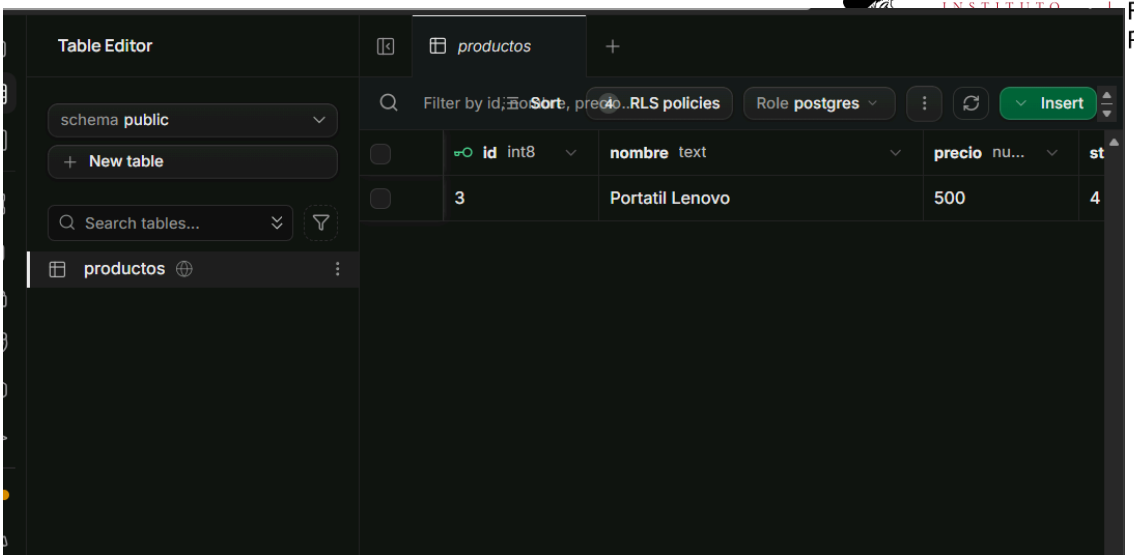
He iniciado sesión como usuario estándar y he comprobado que no puedo acceder a Dashboard ni a Nuevo producto. Al intentar abrir estas páginas, la aplicación me redirige a Productos. También he comprobado que el administrador sí puede acceder a ambas.

Configuración de productos y seguridad en Supabase

He creado la tabla public.productos en Supabase con los campos id, nombre, precio, stock y created_at. El identificador se genera automáticamente y es la clave primaria; nombre, precio y stock son obligatorios y la fecha de creación tiene el valor predeterminado now(). Antes de crearla, he comprobado que no existían tablas en el esquema public.

He activado Row Level Security y he configurado cuatro políticas: lectura para los roles admin y usuario e inserción, actualización y borrado solo para admin. El acceso sin sesión está bloqueado. Los roles se almacenan en app_metadata, administrado desde un entorno autorizado. He probado los permisos en la base de datos y he revertido los registros de prueba. El asesor de seguridad no ha mostrado avisos.

Evidencias de la tabla y sus políticas



He comprobado en public.productos de Supabase que Portatil Lenovo está guardado con precio 500 y stock 4.

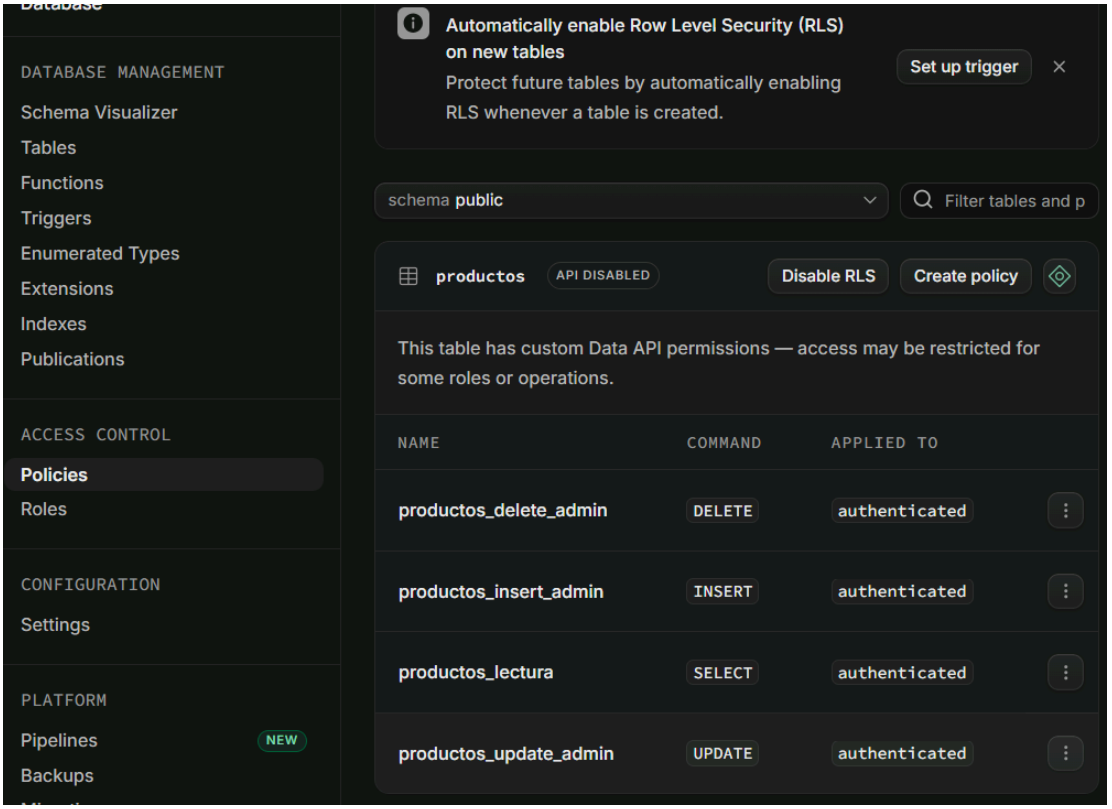


Figura. RLS activado y cuatro políticas para consultar, insertar, actualizar y borrar productos.

Integración de Ionic con la API y autenticación real

He instalado @supabase/supabase-js y he configurado la URL del proyecto y la clave publicable en los entornos de la aplicación. He sustituido el acceso simulado por Supabase Auth y he asignado el rol admin a la cuenta confirmada. Los guards esperan a que se restaure la sesión antes de comprobar el acceso. No he incluido claves secretas ni contraseñas en el cliente.

He conectado el formulario Nuevo producto con la inserción en Supabase y la pantalla Productos con la consulta del catálogo. La aplicación espera la respuesta del servicio antes de confirmar el guardado y muestra los estados de carga y error. He comprobado que la versión integrada compila correctamente y la he ejecutado en localhost:4202.

Código del cliente de Supabase



Figura. Servicio supabase.ts con createClient y las variables de configuración del entorno.

Código de autenticación real

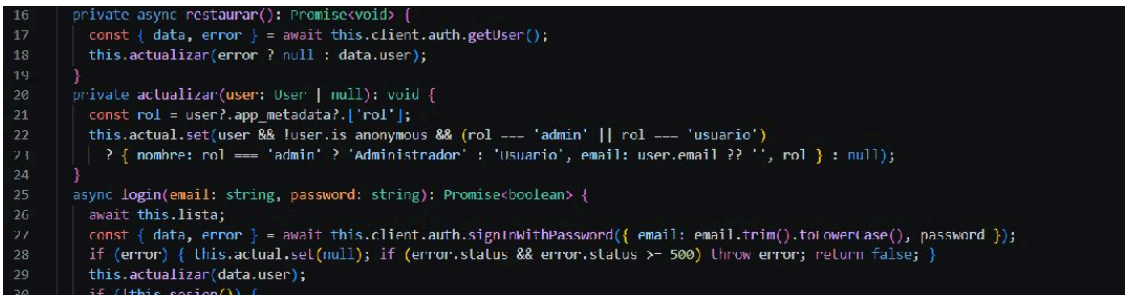


Figura. Extracto de auth.ts con getUser, el rol en app_metadata y signInWithPassword de Supabase Auth.

Comprobación del guardado y consulta de productos

He iniciado sesión con la cuenta administradora y he registrado Portatil Lenovo con un precio de 500 euros y un stock de cuatro unidades. Después de guardarlo, aparece en el listado de Productos, como se observa en la captura.

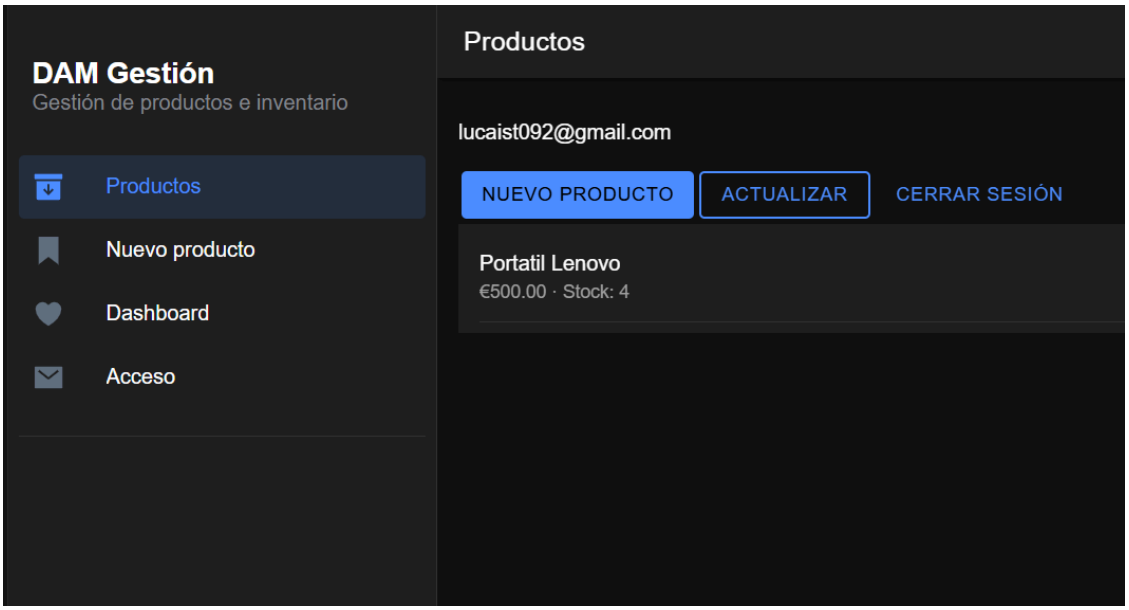


Figura. Producto guardado desde Ionic y mostrado en el listado conectado a Supabase.

He consultado directamente la tabla productos de Supabase y he confirmado que contiene el mismo registro, con el nombre, el precio y el stock indicados. Esta comprobación acredita el almacenamiento real, además de su visualización en la aplicación. Queda pendiente verificar la interfaz con una cuenta usuario de Supabase y completar los componentes reutilizables, el dashboard y el resto de requisitos del enunciado.

Componentes reutilizables y dashboard de inventario

He creado dos componentes visuales independientes. La tabla recibe el catálogo mediante un Input, permite buscar por nombre y emite eventos al cambiar el filtro o seleccionar una fila. La he reutilizado en Productos y en Dashboard. El indicador recibe su título, valor y estado activo y notifica las pulsaciones mediante un Output.

He desarrollado el dashboard protegido para el administrador. A partir de los productos consultados en Supabase, calcula el número de referencias, las unidades, el valor del inventario y los productos con stock bajo. El valor corresponde al precio de venta multiplicado por las unidades; no representa beneficio ni coste de adquisición. He definido stock bajo como cinco unidades o menos, incluyendo los productos agotados.

He incorporado un gráfico de barras SVG con hasta diez productos ordenados por stock y una tabla con los valores. Al pulsar el indicador de stock bajo, el gráfico y la tabla muestran únicamente esos productos. Los indicadores generales conservan el resumen de todo el catálogo. He añadido estados de carga y error, botones accesibles por teclado y una distribución adaptable de los indicadores.

He ejecutado cuatro pruebas automatizadas: catálogo vacío; cálculo del resumen y del umbral de stock bajo; filtrado y selección de productos; y emisión del evento al pulsar un indicador. Todas han pasado. La compilación de producción también ha finalizado correctamente. He comprobado visualmente los indicadores del dashboard y he añadido la captura a continuación. Quedan pendientes el gráfico completo, la tabla, el filtro y la auditoría Lighthouse.

Código de la tabla reutilizable

```
13  export class TablaProductos {
14    readonly productos = input.required<readonly Producto[]>();
15    readonly seleccion = output<Producto>();
16    readonly filtroCambio = output<string>();
17    readonly busqueda = signal('');
18    readonly filtrados = computed(() => this.productos().filter(p => p.nombre.toLocaleLowerCase('es').includes(this.busqueda().toLocaleLo
19    buscar(texto: string): void { this.busqueda.set(texto); this.filtroCambio.emit(texto); }
20  }
```

Figura. Extracto de tabla-productos.ts con el catálogo de entrada y los eventos de selección y filtrado.

Código del indicador reutilizable

```
7   export class Indicador {
8     readonly titulo = input.required<string>();
9     readonly valor = input.required<string>();
10    readonly detalle = input('');
11    readonly activo = input(false);
12    readonly activar = output<void>();
13  }
```

Figura. Extracto de indicador.ts con los Inputs de configuración y el Output activar.

Comprobación de los indicadores del dashboard

He abierto el dashboard de administrador en localhost:4202/dashboard y he comprobado sus indicadores con el producto registrado en Supabase. Se muestra una referencia y cuatro unidades disponibles. El valor del inventario es de 2.000 euros, calculado como 500 euros por cuatro unidades. El indicador de stock bajo muestra un producto, porque sus cuatro unidades están dentro del umbral de cinco o menos.

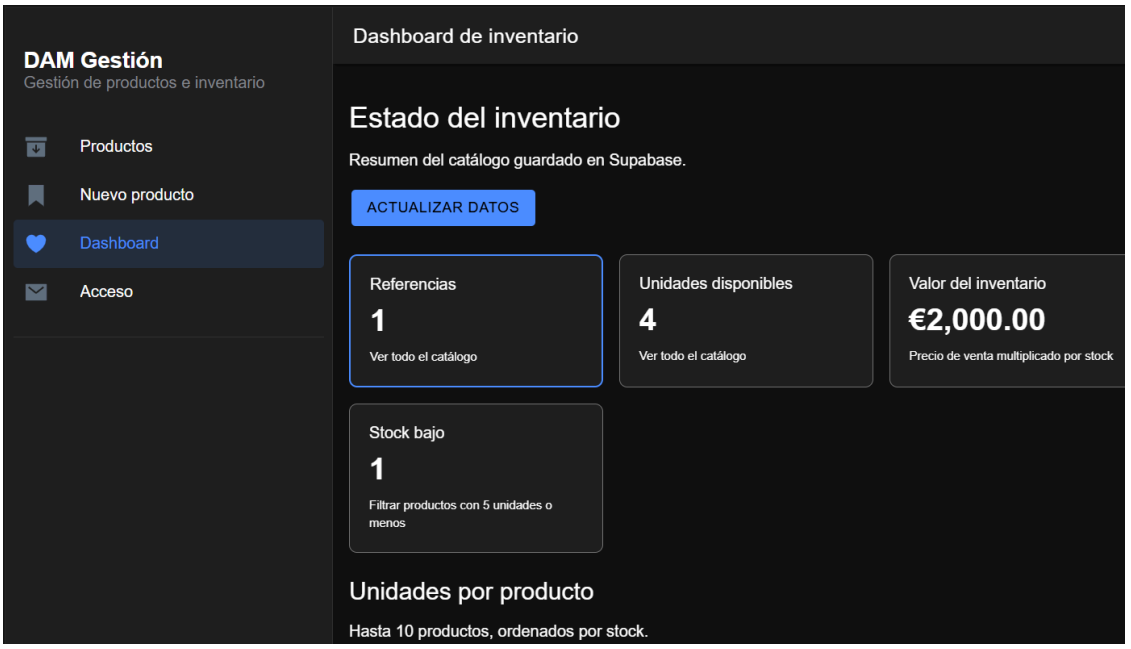


Figura. Dashboard de inventario con los indicadores calculados a partir del catálogo de Supabase.

Esta captura corresponde a la primera comprobación de indicadores. Las verificaciones y evidencias de la versión ampliada se recogen en los apartados siguientes.

Trabajo definitivo en el proyecto original

He integrado los avances en la carpeta original dam-gestion y he fijado localhost:4202 como puerto de desarrollo. He corregido el acceso con Supabase y la actualización del estado del formulario para que un error no deje el botón bloqueado.

Catálogo ampliado y comprobación del dashboard

He añadido diez productos de demostración sin modificar los dos existentes. El catálogo contiene 12 referencias, 232 unidades y un valor de 96.680,90 euros. He incluido existencias agotadas, bajas y normales. He comprobado el listado conectado a Supabase y el filtro de stock bajo, que identifica cinco referencias con hasta cinco unidades.

Informes tabulados PDF y CSV

He implementado informes del catálogo según los filtros del dashboard. El CSV utiliza UTF-8. El PDF incluye una tabla paginada con cabeceras repetidas. He comprobado por pruebas automáticas el contenido del CSV y un PDF de 65 registros. La comprobación de descarga desde el navegador quedó pendiente porque su permiso fue rechazado.

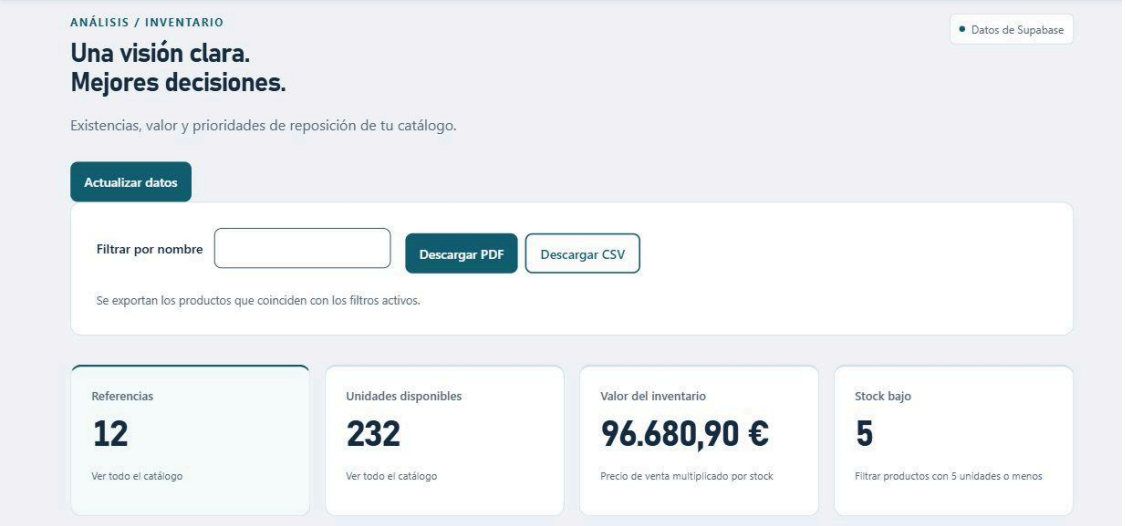


Figura. Dashboard publicado con filtro y controles para generar PDF y CSV.

Diseño adaptable, ayuda y documentación

He unificado el diseño del catálogo, el dashboard y los formularios. He añadido una cabecera con menú para móvil, etiquetas de existencias, foco visible y zoom permitido. He integrado una página de Ayuda y un manual en README. He generado la documentación técnica con Compodoc y he preparado el manifiesto y el service worker de la PWA para producción.



Figura. Catálogo de producción en pantalla móvil, con tarjetas y menú de navegación.



Figura. Manual de usuario integrado en la página Ayuda de producción.

Verificación de acceso y calidad

He creado dos cuentas sintéticas de evaluación, una administradora y otra estándar, separadas de mi cuenta personal. He comprobado el acceso real de ambas y la lectura de los doce productos. En el navegador, el usuario estándar ve solo Productos y Ayuda y es redirigido al intentar abrir Dashboard o Nuevo producto. La API también rechaza sus inserciones mediante RLS y no expone datos sin sesión. He ejecutado 25 pruebas automáticas y el análisis de código, con resultado correcto. Lighthouse sobre la pantalla de acceso de la URL pública ha obtenido 100 en Accessibility, 100 en Best Practices y 95 en Performance. La auditoría local anterior obtuvo 98 en Performance. Esta auditoría no equivale a auditar todas las pantallas autenticadas.

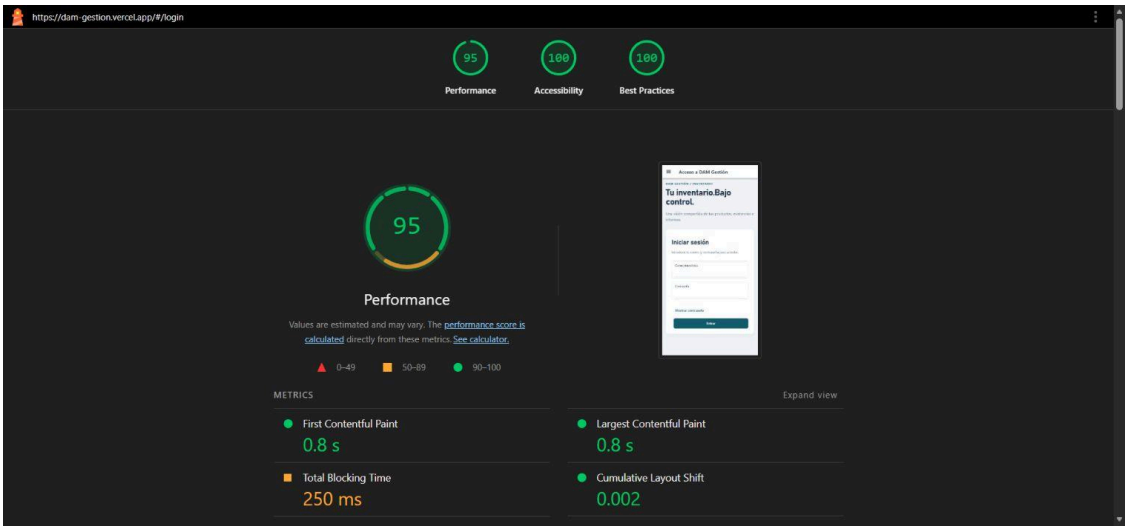


Figura. Auditoría Lighthouse de la URL pública: Performance 95, Accessibility 100 y Best Practices 100.

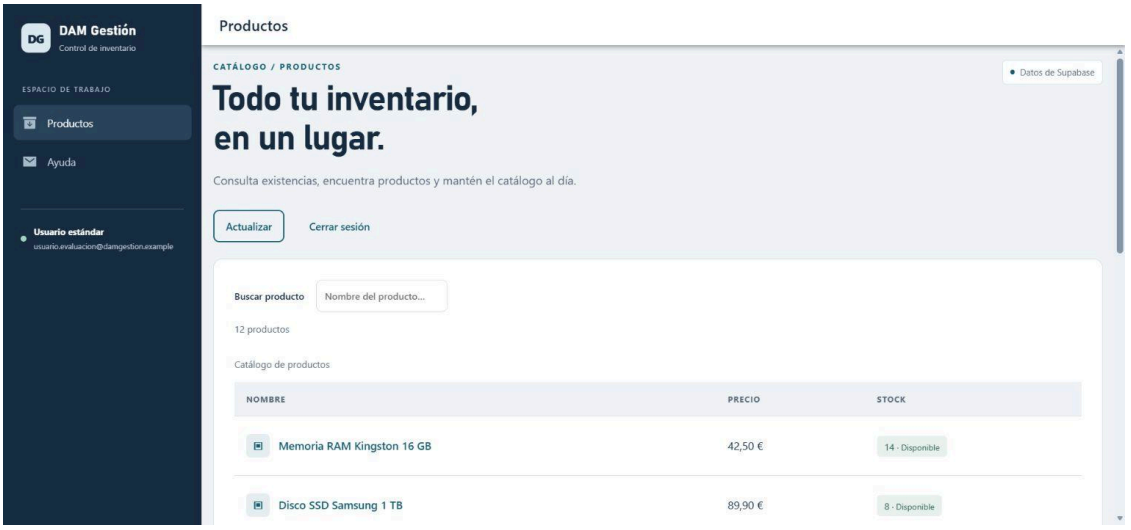


Figura. Usuario estándar en Vercel: consulta el catálogo y no dispone de acciones de alta o edición.

Evidencias de la versión actual

He realizado las siguientes capturas sin el cursor dentro de la zona documentada.

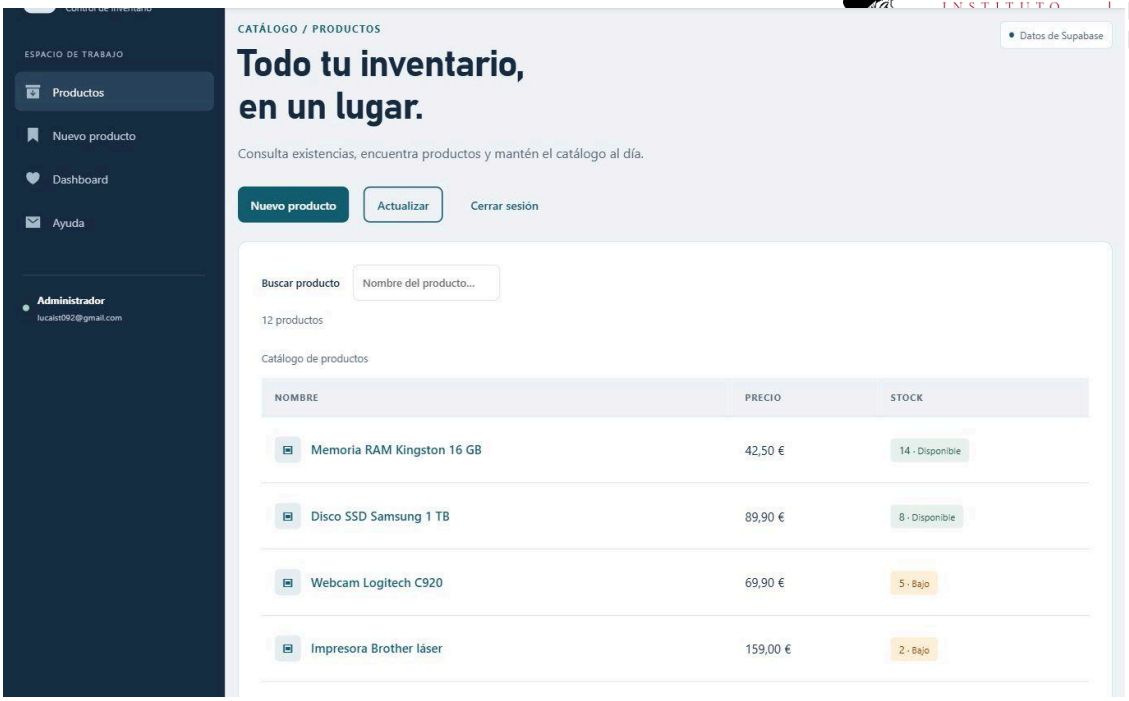


Figura. Catálogo ampliado con doce productos guardados en Supabase.

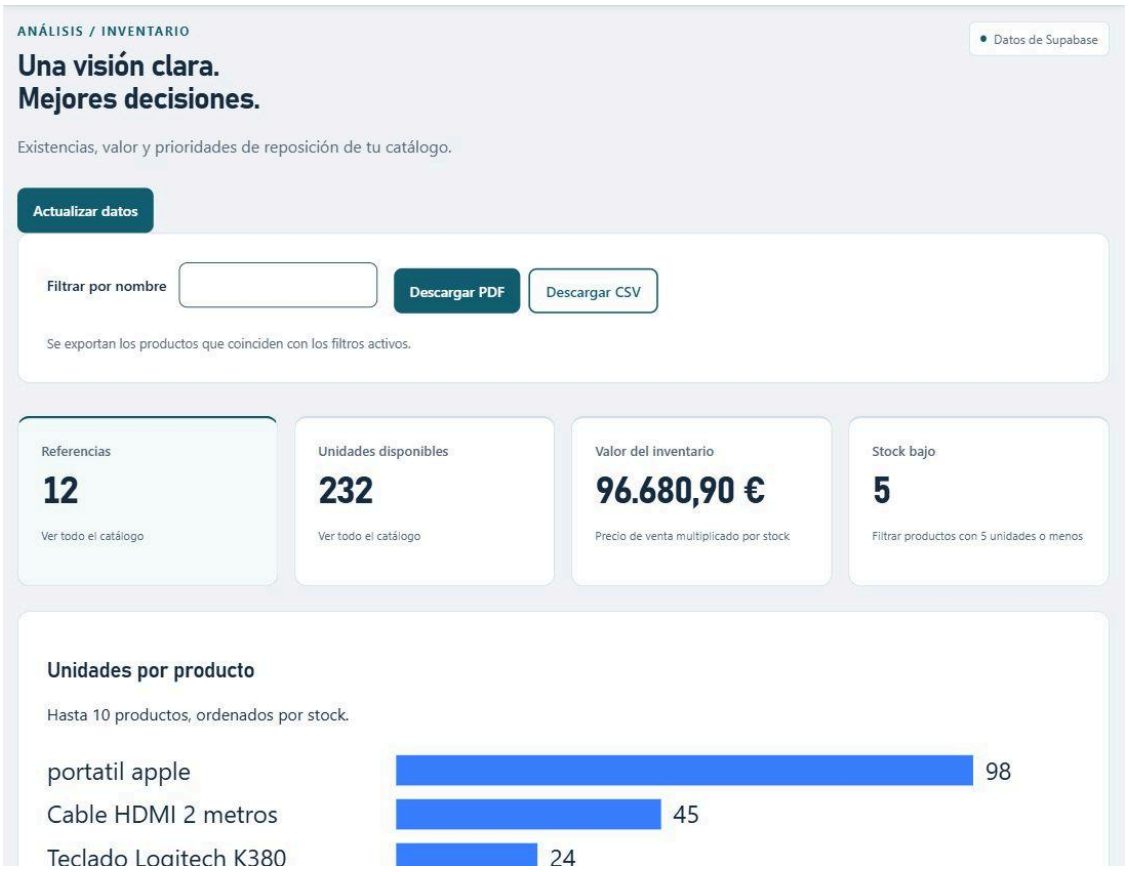


Figura. Dashboard con métricas del catálogo y botones de informes PDF y CSV.



He comprobado que el formulario consulta Supabase y detecta el nombre Portatil Lenovo antes de guardar, evitando un alta repetida**.**

Edición de productos por el administrador

He añadido Editar al catálogo del administrador para modificar nombre, precio y stock. He comprobado el guardado en Supabase y el rechazo de actualizaciones del usuario estándar. Tras la prueba he restaurado Portatil Lenovo a 500 € y stock 4. Las 25 pruebas automáticas, el análisis y la compilación son correctos.



Figura. Formulario con los tres campos del producto editado.

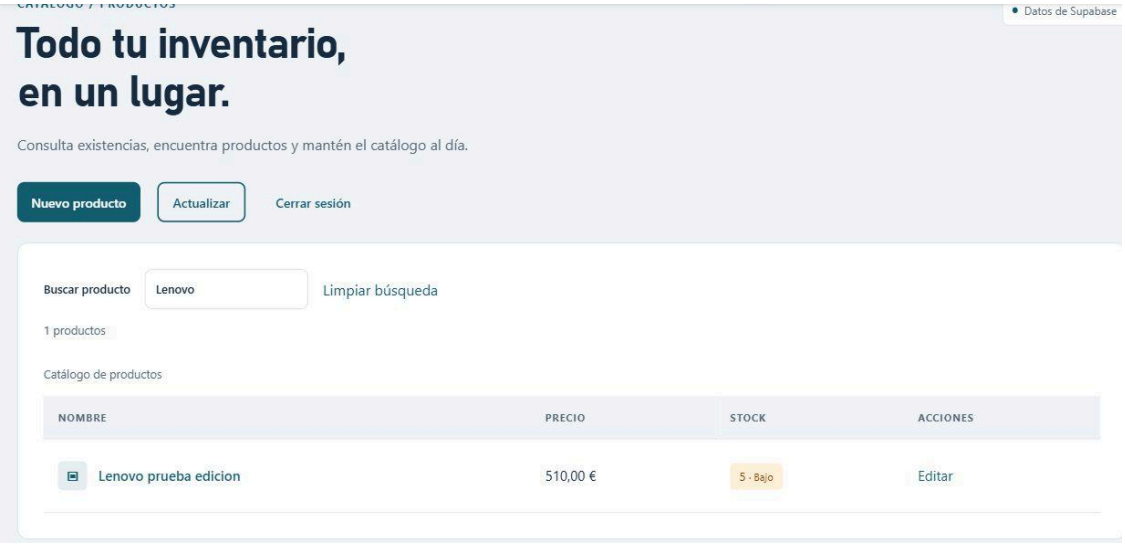


Figura. Catálogo tras guardar el nombre, precio y stock de prueba.

Publicación y enlaces de entrega

He publicado el código completo en el repositorio original de GitHub, conservando el historial anterior y las ramas main, develop y feature. GitHub Actions supera las 25 pruebas, el análisis y la compilación antes de publicar automáticamente en GitHub Pages. Vercel está conectado al mismo repositorio como alojamiento principal. He comprobado el acceso de ambos roles, la lectura del catálogo y el bloqueo de la edición del usuario estándar en Vercel.

Repositorio: https://github.com/lucaslopez72/dam-gestion

Aplicación principal: https://dam-gestion.vercel.app/#/login

Alternativa: https://lucaslopez72.github.io/dam-gestion/

Compodoc: https://lucaslopez72.github.io/dam-gestion/documentacion/index.html

Manual: https://github.com/lucaslopez72/dam-gestion#manual-de-usuario

Lighthouse: https://lucaslopez72.github.io/dam-gestion/evidencias/lighthouse-vercel.html

Despliegue: https://github.com/lucaslopez72/dam-gestion/actions

Las credenciales de las dos cuentas de evaluación se incluyen únicamente en el PDF privado para el docente. La versión pública de esta memoria no contiene contraseñas.

5. CONCLUSIONES

He completado y publicado la aplicación desde el proyecto original, con acceso real por roles, catálogo de Supabase, componentes reutilizables, dashboard, validación asíncrona, simulación documental e informes PDF y CSV. He comprobado la compilación, 25 pruebas y la auditoría de acceso. He preparado la documentación y la configuración de integración continua. He publicado el repositorio y la aplicación, comprobado sus enlaces e incorporado las evidencias a esta memoria. El PDF privado de entrega reúne los enlaces, las cuentas de evaluación y esta memoria.

6. LÍNEAS DE INVESTIGACIÓN FUTURAS

Como continuación, podría incorporar borrado con confirmación, una fuente NoSQL real y registro de cambios. Estas mejoras quedan fuera de la versión local descrita y no las presento como realizadas.

7. BIBLIOGRAFÍA

Documentación oficial consultada el 29 de octubre de 2026:

Angular. Formularios y validación: https://angular.dev/guide/forms/form-validation

Angular. Service workers y PWA: https://angular.dev/ecosystem/service-workers/getting-started

Supabase. Autenticación: https://supabase.com/docs/guides/auth

Supabase. Seguridad de la API: https://supabase.com/docs/guides/api/securing-your-api

Compodoc. Uso y generación de documentación: https://compodoc.app/guides/usage.html

Google. Lighthouse: https://developer.chrome.com/docs/lighthouse/overview

jsPDF. Documentación de generación de PDF: https://artskydj.github.io/jsPDF/docs/jsPDF.html

8. ANEXOS

Conservo como evidencias el informe HTML y JSON de Lighthouse, el resultado de las pruebas automatizadas y la verificación del acceso por roles mediante la API real de Supabase. El manual de usuario está integrado en README y en la página Ayuda; Compodoc genera la documentación técnica.

Las dos cuentas sintéticas de evaluación se guardan en un archivo privado fuera del repositorio. El paquete final de entrega debe contener únicamente un PDF con enlaces al repositorio, a la aplicación publicada, a la documentación y las evidencias exigidas. Las URLs públicas comprobadas se recogen en el apartado Publicación y enlaces de entrega.

9. OTROS PUNTOS

En este apartado recojo las decisiones y dificultades de la integración.

- Aportaciones personales: He integrado las distintas fases en una única aplicación de inventario.
- Retos profesionales: He comprobado la comunicación entre la interfaz, la autenticación y las políticas de la base de datos.
- Retos personales: He consolidado los avances en el proyecto original y he separado lo verificado de lo pendiente.
- Agradecimientos: A las fuentes oficiales de documentación utilizadas como apoyo técnico.